

# miniOS

**the minimum you will need for a bootable OS**

2s2026

Gabriela Bittencourt, João Pedro Leôncio and Vinícius Peixoto

# Contents



|     |                                   |    |
|-----|-----------------------------------|----|
| 1   | minimal Operational Systems ..... | 3  |
| 1.a | Examples .....                    | 4  |
| 1.b | Our project .....                 | 5  |
| 2   | Starter files .....               | 6  |
| 2.a | kernel.c .....                    | 7  |
| 2.b | entry.S .....                     | 9  |
| 2.c | linker.ld .....                   | 12 |
| 3   | Let's dive in .....               | 13 |
| 4   | Next steps .....                  | 14 |

# 1 minimal Operational Systems

# Examples



- minix: Tanenbaum's project, explained on "Modern Operational Systems" book
- xv6: Developed on MIT for educational purpose
  - website: <https://pdos.csail.mit.edu/6.1810/2025/xv6.html>
  - book: <https://pdos.csail.mit.edu/6.1810/2025/xv6/book-riscv-rev5.pdf>
  - playlist on youtube: <https://www.youtube.com/watch?v=fWUJKH0RNFE&list=PL58sEyo8wOZRpDZCa4LQIfbjPz05PDpWF>
- RISC-V Bare Bones: A 'hello world'-sized OS
  - [https://wiki.osdev.org/RISC-V\\_Bare\\_Bones](https://wiki.osdev.org/RISC-V_Bare_Bones)

# Our project



QEMU - quick emulator

- <https://www.qemu.org/>

RISC-V

- <https://riscv.github.io/riscv-isa-manual/snapshot/privileged>

## 2 Starter files

# kernel.c



- The starting point of the OS
- Definition of a main `kmain` in the case of kernel

```
void kmain(void) {  
    print("Hello world!\r\n");  
    while(1) {  
  
    }  
    return;  
}
```

## kernel.c (ii)



```
unsigned char * uart = (unsigned char *)0x10000000;
void putchar(char c) {
    *uart = c;
    return;
}

void print(const char * str) {
    while(*str != '\0') {
        putchar(*str);
        str++;
    }
    return;
}
```



## entry.S



Assembly code to set up the *stack* reset *satp*, clear *.bss* section and **jump to kernel main**.

### **stack**

Points the next instruction to be executed. To execute a program (or a piece of code), the code has to be loaded in memory (instructions are 'stack' the stack).

### **satp**

*supervisor address translation and protection register* – in the future we will use to set up page table

### **.bss**

*block starting symbol* – portion of the code that contains statically allocated variables that are declared but not have been assigned a value yet.

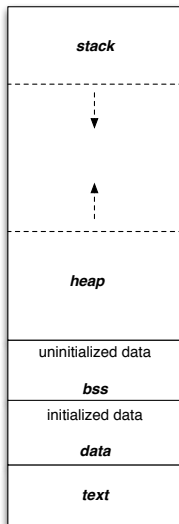


Figure 1: Typical layout of a simple computer's program memory [<https://en.wikipedia.org/wiki/.bss>]

## entry.S (iii)



```
(...)  
csrw satp, zero  
  
la sp, stack_top  
  
la t5, bss_start  
la t6, bss_end  
bss_clear:  
sd zero, (t5)  
addi t5, t5, 8  
bltu t5, t6, bss_clear  
  
(...)  
/* Jump to kernel! */  
tail kmain
```

## linker.ld



- Specify where our kernel will be loaded at.
- For this example we will use the start of the RAM as our load address.

**3 Let's dive in**

## 4 Next steps